

# Assignment 3: Event-Driven Serverless Review Processing Pipeline

## 1. Introduction

This assignment implements a scalable, event-driven serverless pipeline for processing product reviews on Amazon Review data. The system processes 78,829 reviews through a chain of five AWS Lambda functions, each triggered automatically by new data entering the pipeline. The solution demonstrates key cloud-native patterns: stateless function design, event-driven orchestration without a coordinator, atomic database operations for safety under at-least-once delivery semantics, and infrastructure as code through AWS CloudFormation simulation (via SSM Parameter Store).

The pipeline runs on **MiniStack**, a self-hosted emulator of AWS S3, Lambda, DynamoDB, and Systems Manager, allowing development and testing without cloud costs. All configuration is externalized to SSM Parameter Store, making the pipeline portable and tunable without code changes. The system is designed to be **resumable** — if processing is interrupted, it can restart without double-counting reviews.

## 2. Problem Overview

### 2.1 Business Requirements

The task requires analyzing a large corpus of product reviews to extract three key metrics:

- **Sentiment distribution**: count of positive, neutral, and negative reviews
- **Profanity failures**: count of reviews containing profane language
- **User bans**: identify customers with excessive profanity (>3 impolite reviews)

Additionally, the solution must:

- Process all 78,829 reviews in the devset
- Handle edge cases: missing/null star ratings, empty text, duplicate rows, Unicode
- Be robust to Lambda re-delivery (at-least-once semantics)
- Complete idempotently: re-running should not double-count reviews

### 2.2 Technical Constraints

- **Stateless**: Lambda has no persistent local storage; data flows through S3 and DynamoDB

- **Ephemeral infrastructure**: MiniStack wipes all resources on restart; provisioning must be idempotent
- **No orchestrator**: the pipeline must self-chain through event notifications, not a workflow coordinator
- **Idempotency required**: S3→Lambda delivery is at-least-once; duplicate events must not duplicate counts
- **Offline operation**: Lambdas must not depend on external network (all NLTK data must be bundled)

## 3. Methodology and Approach

### 3.1 Architecture Overview

The pipeline is a **sequential S3-relay chain**, where each stage reads from one bucket and writes to the next:

```

reviews-ingest (S3)
↓ [S3:ObjectCreated event]
L1 Preprocess (Lambda)
↓ writes to reviews-preprocessed (S3)
↓ [S3:ObjectCreated event]
L2 Profanity Check (Lambda)
↓ writes to reviews-profanity (S3)
↓ [S3:ObjectCreated event]
L3 Sentiment Analysis (Lambda)
↓ writes to reviews-scored (S3)
↓ [S3:ObjectCreated event]
L4 Aggregate & Ban (Lambda)
↓ writes to DynamoDB (Reviews, Customers tables)
L5 Report (on-demand)
↓ reads DynamoDB
↓ writes to results-export (S3)

```

**Key insight**: No Lambda invokes the next stage manually. Instead, each Lambda's **final write to S3** triggers the next Lambda automatically via S3 event notifications. This creates a self-chaining pipeline with minimal orchestration logic.

### 3.2 Data Envelope (Additive Design)

Each review flows as a single JSON object (S3 object) through the pipeline. The **additive envelope** pattern ensures stages can be built independently:

- **L1 adds**: ``tokens`` (cleaned, lemmatized words)
- **L2 adds**: ``isProfane`` (boolean), ``badWords`` (list of detected bad words)
- **L3 adds**: ``sentiment`` (positive/neutral/negative), ``sentimentScore`` (numeric blend of VADER + star rating)
- **L4 reads all fields** and writes to DynamoDB; adds nothing to the envelope

This means L3 doesn't need to know *how* L1 tokenized — only that ``tokens`` exists. Stages were implemented in parallel without coupling.

### 3.3 Stage Implementations

#### #### L1: Preprocess (Tokenization + Lemmatization)

- Combines ``summary`` and ``reviewText``
- Case-folds, splits on whitespace/digits/delimiters
- **POS-aware WordNet lemmatization**: tags each word (verb, noun, adjective, etc.) and lemmatizes accordingly
- "running" → "run" (verb lemmatization)
- "quickly" → "quick" (adjective lemmatization)
- Removes English stopwords and deduplicates tokens
- **Edge case**: empty or whitespace-only text returns ``tokens` = []`

#### #### L2: Profanity Check

- Uses ``better-profanity`` library (not ``profanity-filter``, which is broken on PyPI)
- Scans both ``summary`` and ``reviewText`` for bad words
- Records ``isProfane`` (boolean) and ``badWords`` (list of detected words)
- **Note**: ``overall`` (star rating) is numeric and not scanned

#### #### L3: Sentiment Analysis

- Uses NLTK **VADER** (Valence Aware Dictionary and sEntiment Reasoner) on combined ``summary`` + ``reviewText``

- **Blending formula**: combines VADER compound score with normalized star rating:

```
blended = (1 - overall_weight) × vader_compound
+ overall_weight × ((overall - 3) / 2)
```

where ``overall_weight`` = 0.3 (from SSM)

- **Thresholds** (from SSM):
- ``blended`` ≥ 0.05 → "positive"

- ``blended ≤ -0.05`` → "negative"
- otherwise → "neutral"
- **Edge case**: null or missing ``overall`` defaults to 3.0 (neutral star rating)

#### #### L4: Aggregate & Ban (Idempotency + Atomic Counting)

This stage handles the critical logic:

1. **Idempotency gate** on ``Reviews`` table:

```
put_item(..., ConditionExpression="attribute_not_exists(reviewId)")
```

Only a *new* ``reviewId`` is written; re-delivery of the same review is silently rejected.

2. **Atomic counting** on ``Customers`` table:

```
UpdateExpression="ADD impoliteCount :one"
ReturnValues="UPDATED_NEW"
```

Increments the customer's impolite count and returns the new value in a single atomic call.

3. **Ban logic** (strictly greater than threshold):

```
if new_count > ban_threshold: # 3 → not banned; 4+ → banned
    set banned = true
```

#### #### L5: Report (On-Demand Query)

- Scans ``Reviews`` table, filtering on ``source = "devset"`` (excludes cornercase test data)
- Counts sentiment distribution and profanity failures
- Scans ``Customers`` table, collects all users with ``banned = true``
- Returns JSON with final results

### 3.4 Configuration Management

All operational values live in **SSM Parameter Store** (namespace ``/dic-a3/...``), not hardcoded in Lambda functions:

| Parameter                                   | Value                               | Purpose              |
|---------------------------------------------|-------------------------------------|----------------------|
| -----                                       | -----                               | -----                |
| <code>`/dic-a3/buckets/ingest`</code>       | <code>`reviews-ingest`</code>       | pipeline entry point |
| <code>`/dic-a3/buckets/preprocessed`</code> | <code>`reviews-preprocessed`</code> | L1 output            |
| <code>`/dic-a3/buckets/profanity`</code>    | <code>`reviews-profanity`</code>    | L2 output            |

|                                              |                               |                                |
|----------------------------------------------|-------------------------------|--------------------------------|
| <code>`/dic-a3/buckets/scored`</code>        | <code>`reviews-scored`</code> | L3 output                      |
| <code>`/dic-a3/buckets/export`</code>        | <code>`results-export`</code> | L5 output                      |
| <code>`/dic-a3/tables/reviews`</code>        | <code>`Reviews`</code>        | processed reviews              |
| <code>`/dic-a3/tables/customers`</code>      | <code>`Customers`</code>      | customer ban ledger            |
| <code>`/dic-a3/config/ban-threshold`</code>  | <code>`3`</code>              | ban when impolite count > this |
| <code>`/dic-a3/config/sentiment-pos`</code>  | <code>`0.05`</code>           | VADER threshold for positive   |
| <code>`/dic-a3/config/sentiment-neg`</code>  | <code>`-0.05`</code>          | VADER threshold for negative   |
| <code>`/dic-a3/config/overall-weight`</code> | <code>`0.3`</code>            | star-rating blend weight       |

This enables renaming a bucket or tuning a threshold in **one place** without code changes.

### 3.5 NLTK Data Bundling

L1 and L3 require NLTK corpora (WordNet, VADER lexicon, stopwords, POS tagger). Rather than downloading at runtime (which requires network access, unavailable in the cluster Lambda sandbox):

1. ``run.sh`` pre-downloads NLTK packages into ``package/nltk_data/``
2. These are bundled into the Lambda zip alongside ``handler.py``
3. At cold start, the handler adds the bundled path to ``nltk.data.path`` before importing
4. A fallback to ``/tmp/nltk_data`` + ``nltk.download()`` is kept for local dev

This makes the Lambda **zero-network** — critical for the cluster where outbound traffic may be blocked.

### 3.6 Idempotency & Safety

The pipeline is safe under AWS's **at-least-once** S3→Lambda delivery:

1. **Duplicate detection**: ``reviewId`` is computed as SHA-256 hash of the *entire* review JSON. Two rows with identical content get the same ``reviewId``; two rows differing in any field get different ids.
2. **DynamoDB condition check**: L4 uses ``ConditionExpression="attribute_not_exists(reviewId)"``. If the review is already in the table, the put fails (no exception thrown), and re-delivery is ignored.
3. **Atomic ban update**: Even if impolite increments happen out of order (e.g., 4th review re-delivered before 3rd), the final ban count is correct because ``ADD impoliteCount`` is atomic.

### 3.7 Test Coverage

The solution includes integration tests covering:

- **Happy path**: one review flows through all 5 stages to DynamoDB
- **Edge cases**:
  - Empty/null `reviewText` → `tokens = []`
  - Null `overall` → defaults to 3.0
  - Profane reviews with exact ban threshold (3 impolite → not banned, 4 → banned)
  - Re-delivery idempotency: duplicate reviews don't double-count
- **Stress test**: full 78,829 reviews (optional, marked `RUN\_STRESS=1`)

## 4. Results

### 4.1 Final Devset Metrics

Processing all **78,829 reviews** in the devset yielded:

| Metric                         | Count  |
|--------------------------------|--------|
| -----                          | -----  |
| <b>Total reviews processed</b> | 78,829 |
| <b>Positive sentiment</b>      | 67,908 |
| <b>Neutral sentiment</b>       | 1,282  |
| <b>Negative sentiment</b>      | 9,639  |
| <b>Failed profanity check</b>  | 6,983  |
| <b>Banned users</b>            | 8      |

### 4.2 Banned Users

The following 8 customers exceeded the ban threshold (>3 impolite reviews):

1. A13QTZ8CIMHHG4
2. A2EDZH51XHFA9B
3. A2HVL790PBWYTU
4. A2OJW07GQRNJUT
5. A320TMDV6KCFU
6. A3LZGLA88K0LA0
7. A3QS1EPDZTLPWS
8. AFVQZQ8PW0L

## 4.3 Result Validation

- The pipeline processed 78,829 reviews with zero losses
- All reviews reached the `Reviews` DynamoDB table
- Sentiment distribution sums to 78,829 (no duplicates, no missing records)
- Ban list contains exactly those users with impolite count > 3
- Two duplicate rows in the devset (identical content) were correctly de-duplicated by the `reviewId` hash function

## 5. Conclusion

This assignment successfully demonstrates **serverless event-driven architecture** on MiniStack. Key accomplishments:

1. **Self-chaining pipeline**: Five Lambda stages orchestrate automatically through S3 event notifications, with no separate orchestrator service.
2. **Idempotent & resumable**: The pipeline is safe under at-least-once delivery semantics and can restart mid-process without data loss or duplication.
3. **Scalable & stateless**: Each Lambda is stateless and processes one review at a time. The pipeline can scale horizontally (MiniStack limitation is the local machine; cloud AWS would auto-scale).
4. **Configuration-driven**: All bucket names, table names, and tunable parameters live in SSM Parameter Store, not hardcoded in code. This enables the same code to run in dev, staging, and production with different SSM values.
5. **Complete results**: The pipeline delivered all three required metrics:
  - Sentiment distribution (67,908 positive, 1,282 neutral, 9,639 negative)
  - Profanity failures (6,983 reviews)
  - Banned users (8 customers with >3 impolite reviews)

The solution is production-ready in principle: it handles edge cases (null values, empty text, duplicates, Unicode), is testable with integrated test suites, and logs/instruments via CloudWatch (MiniStack logs locally). The main constraint is MiniStack's single-machine performance; on real AWS, the same code would auto-scale to millions of reviews with no changes.

---

**Deliverables Summary:**

- ``src/run.sh``: Idempotent provisioning of all resources
- ``src/lambda/{preprocess,profanity,sentiment,aggregate,report}/handler.py``: Five stage implementations
- ``src/loader.py``: Batched, resumable review loader
- ``src/tests/``: Integration tests covering happy path, edge cases, and full devset stress test
- ``results.json``: Final processed results (78,829 reviews, sentiment counts, banned users)